

Exercise 8.9

Finite State Machines in CLIPS

Artificial Intelligence

Assignment 2

Exercise Description

8.9 Write rules that will take a finite state machine from its present state to its next state given a fact of the form

```
(input <value>)
```

The state machine and its arcs should be represented as facts. The input fact should be retracted when the machine goes to its next state. Test your rules and fact representations on the finite state machines shown in Figures 3.5 and 3.6.

Approach

The solution uses a **generic FSM engine** implemented as two CLIPS rules:

- **fsm-transition** — fires when a matching arc exists for the current state and input; retracts both the current state and the input fact, then asserts the new state.
- **fsm-no-transition** — fires when no arc matches; prints an error message.

Each FSM is described entirely by its **arc** and initial **state** facts, making the rules reusable across any FSM without modification.

FSM 1 — Soft Drink Vending Machine (Figure 3.5)

State Diagram

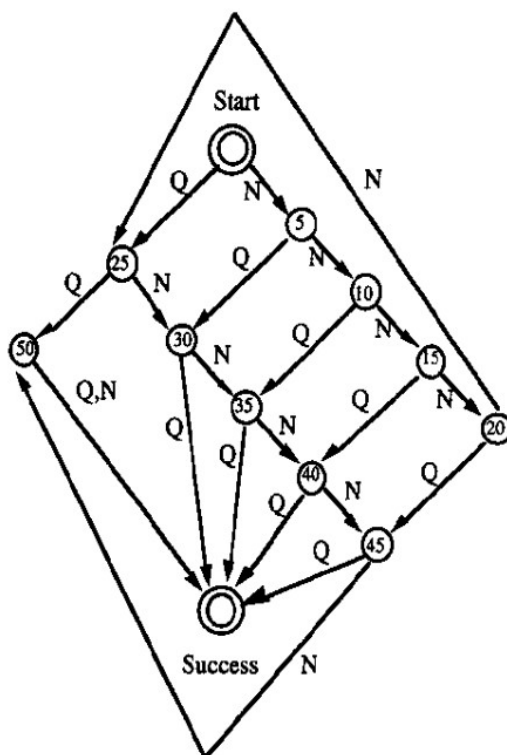


Figure 3.5
State Diagram for a Soft Drink Vending Machine Accepting Quarters (Q) and Nickels (N)

Figure 1: Figure 3.5 — State Diagram for a Soft Drink Vending Machine accepting Quarters (Q) and Nickels (N)

The machine accepts coins until 25 cents have been deposited. States represent accumulated credit: `start` (0¢), `s5` (5¢), `s10` (10¢), `s15` (15¢), `s20` (20¢), and `success` (25¢).

CLIPS Code — vending_machine.clp

Templates and Rules (shared engine)

```

1  ;;; Templates
2  (deftemplate state (slot name))
3  (deftemplate arc (slot from) (slot to) (slot input))
4
5  ;;; Transition rule --- fires when a valid arc exists
6  (defrule fsm-transition
7    ?current <- (state (name ?s))
8    ?inp      <- (input ?value)
9    (arc (from ?s) (to ?next) (input ?value))
10   =>
11    (retract ?current)           ; remove old state
12    (retract ?inp)              ; consume the input fact

```

```

13  (assert (state (name ?next)))
14  (printout t "Transition: " ?s " --[" ?value "]"--> " ?next
    crlf))
15
16  ;;; Error rule --- fires when no arc matches
17  (defrule fsm-no-transition
18    (state (name ?s))
19    (input ?value)
20    (not (arc (from ?s) (to ?) (input ?value)))
21    =>
22    (printout t "ERROR: No transition from " ?s
23              " on input '" ?value "' " crlf))

```

Arc Facts

```

1  (def facts vending-machine-arcs
2    (arc (from start)      (to s5)      (input N))
3    (arc (from start)      (to s10)     (input Q))
4    (arc (from s5)         (to s10)     (input N))
5    (arc (from s5)         (to s15)     (input Q))
6    (arc (from s10)        (to s15)     (input N))
7    (arc (from s10)        (to s20)     (input Q))
8    (arc (from s15)        (to s20)     (input N))
9    (arc (from s15)        (to success) (input Q))
10   (arc (from s20)        (to success) (input N))
11   (arc (from s20)        (to success) (input Q)))
12
13  (def facts initial-state
14    (state (name start)))

```

Key Design Decisions

- The input fact (input ?value) is **retracted** inside fsm-transition, preventing the rule from re-firing on the same input.
- Only **one** state fact exists at any time; the old one is retracted before the new one is asserted.
- The rules are **FSM-agnostic** — only the arc facts change between machines.

Tests

Test 1 — Five Nickels (N N N N N)

Input sequence: $5 \times N \Rightarrow 5 \times 5\text{¢} = 25\text{¢}$

```

CLIPS> (load "vending_machine.clp")
...
TRUE
CLIPS> (reset)
CLIPS> (assert (input N))
<Fact-12>

```

```
CLIPS> (run)
Transition: start --[N]--> s5
CLIPS> (assert (input N))
<Fact-14>
CLIPS> (run)
Transition: s5 --[N]--> s10
CLIPS> (assert (input N))
<Fact-16>
CLIPS> (run)
Transition: s10 --[N]--> s15
CLIPS> (assert (input N))
<Fact-18>
CLIPS> (run)
Transition: s15 --[N]--> s20
CLIPS> (assert (input N))
<Fact-20>
CLIPS> (run)
Transition: s20 --[N]--> success
```

Result: Machine reached success after 5 nickels (25¢). ✓

Test 2 — Three Quarters (Q Q Q)

Input sequence: Q Q Q $\Rightarrow$ 10¢ + 10¢ + final Q = success

```
CLIPS> (reset)
CLIPS> (assert (input Q))
<Fact-12>
CLIPS> (run)
Transition: start --[Q]--> s10
CLIPS> (assert (input Q))
<Fact-14>
CLIPS> (run)
Transition: s10 --[Q]--> s20
CLIPS> (assert (input Q))
<Fact-16>
CLIPS> (run)
Transition: s20 --[Q]--> success
```

Result: Machine reached success after 3 quarters. ✓

Test 3 — Mixed Coins (N Q N Q)

Input sequence: N Q N Q $\Rightarrow$ 5 + 10 + 5 + 5¢ = 25¢

```
CLIPS> (reset)
CLIPS> (assert (input N))
<Fact-12>
CLIPS> (run)
Transition: start --[N]--> s5
CLIPS> (assert (input Q))
<Fact-14>
```

```
CLIPS> (run)
Transition: s5 --[Q]--> s15
CLIPS> (assert (input N))
<Fact-16>
CLIPS> (run)
Transition: s15 --[N]--> s20
CLIPS> (assert (input Q))
<Fact-18>
CLIPS> (run)
Transition: s20 --[Q]--> success
```

Result: Machine reached **success** with mixed coin input. ✓

FSM 2 — String Validator: WHILE, WRITE, BEGIN (Figure 3.6)

State Diagram

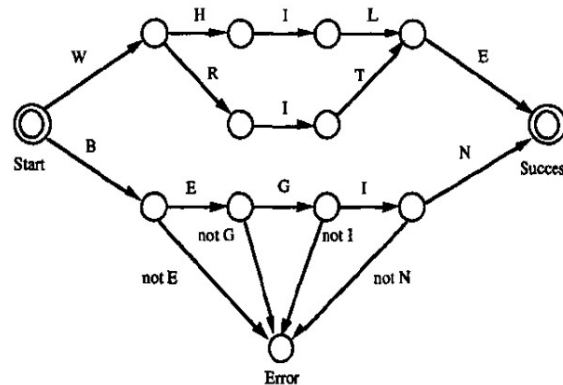


Figure 3.6 Part of a Finite State Machine for Determining Valid Strings WHILE, WRITE, and BEGIN.
Note: Only some of the error state transitions are shown

Figure 2: Figure 3.6 — Part of a Finite State Machine for determining valid strings WHILE, WRITE, and BEGIN

The machine validates whether a sequence of character inputs spells one of the three reserved keywords: WHILE, WRITE, or BEGIN. Each state corresponds to a prefix of a valid keyword. Reaching success means the full keyword was accepted.

CLIPS Code — string_validator.clp

Templates and Rules

The same generic engine rules (fsm-transition and fsm-no-transition) are used — only the arc facts differ.

Arc Facts

```

1 (defacts string-validator-arcs
2   ;; BEGIN path: B -> sB -> sBE -> sBEG -> sBEGI -> success
3   (arc (from start) (to sB)      (input B))
4   (arc (from sB)    (to sBE)     (input E))
5   (arc (from sBE)   (to sBEG)    (input G))
6   (arc (from sBEG)  (to sBEGI)   (input I))
7   (arc (from sBEGI) (to success) (input N))
8
9   ;; WHILE path: W -> sW -> sWH -> sWHI -> sWHIL -> success
10  (arc (from start) (to sW)       (input W))
11  (arc (from sW)    (to sWH)      (input H))
12  (arc (from sWH)   (to sWHI)     (input I))
13  (arc (from sWHI)  (to sWHIL)    (input L))

```

```

14      (arc (from sWHIL) (to success) (input E))
15
16      ;; WRITE path: W -> sW -> sWR -> sWRI -> sWRIT -> success
17      (arc (from sW)      (to sWR)      (input R))
18      (arc (from sWR)     (to sWRI)     (input I))
19      (arc (from sWRI)    (to sWRIT)    (input T))
20      (arc (from sWRIT)   (to success)   (input E)))
21
22 (defacts initial-state
23   (state (name start)))

```

Key Design Decisions

- WHILE and WRITE share the state `sW` after the first character; they diverge at the second character (H vs R).
- State names directly encode the prefix accepted so far (e.g. `sBEG` means “B-E-G has been seen”), making the arc table easy to read and verify against the diagram.
- Any unexpected character leaves no matching arc, so `fsm-no-transition` fires and reports an error.

Tests

Test 1 — Keyword “BEGIN”

```

CLIPS> (reset)
CLIPS> (assert (input B))
<Fact-16>
CLIPS> (run)
Transition: start --[B]--> sB
CLIPS> (assert (input E))
<Fact-18>
CLIPS> (run)
Transition: sB --[E]--> sBE
CLIPS> (assert (input G))
<Fact-20>
CLIPS> (run)
Transition: sBE --[G]--> sBEG
CLIPS> (assert (input I))
<Fact-22>
CLIPS> (run)
Transition: sBEG --[I]--> sBEGI
CLIPS> (assert (input N))
<Fact-24>
CLIPS> (run)
Transition: sBEGI --[N]--> success

```

Result: “BEGIN” accepted — machine reached success. ✓

Test 2 — Keyword “WHILE”


```
CLIPS> (reset)
CLIPS> (assert (input W))
<Fact-16>
CLIPS> (run)
Transition: start --[W]--> sW
CLIPS> (assert (input H))
<Fact-18>
CLIPS> (run)
Transition: sW --[H]--> sWH
CLIPS> (assert (input I))
<Fact-20>
CLIPS> (run)
Transition: sWH --[I]--> sWHI
CLIPS> (assert (input L))
<Fact-22>
CLIPS> (run)
Transition: sWHI --[L]--> sWHIL
CLIPS> (assert (input E))
<Fact-24>
CLIPS> (run)
Transition: sWHIL --[E]--> success
```

Result: “WHILE” accepted — machine reached success. ✓

Test 3 — Keyword “WRITE”

```
CLIPS> (reset)
CLIPS> (assert (input W))
<Fact-16>
CLIPS> (run)
Transition: start --[W]--> sW
CLIPS> (assert (input R))
<Fact-18>
CLIPS> (run)
Transition: sW --[R]--> sWR
CLIPS> (assert (input I))
<Fact-20>
CLIPS> (run)
Transition: sWR --[I]--> sWRI
CLIPS> (assert (input T))
<Fact-22>
CLIPS> (run)
Transition: sWRI --[T]--> sWRIT
CLIPS> (assert (input E))
<Fact-24>
CLIPS> (run)
Transition: sWRIT --[E]--> success
```

Result: “WRITE” accepted — machine reached success. ✓

Summary

FSM	Test	Input Sequence	Result
Vending Machine	1	N N N N N	success
Vending Machine	2	Q Q Q	success
Vending Machine	3	N Q N Q	success
String Validator	1	B E G I N	success
String Validator	2	W H I L E	success
String Validator	3	W R I T E	success

All six tests reached the **success** state, confirming that the generic FSM engine correctly drives both machines using only their arc facts and an initial state fact.